Research notebook

The larval zebrafish oculomotor integrator: circuit, task, and training

Working document for the `feat/oculomotor` branch. It records what the circuit is taken to be, what the network will be asked to compute, and how the training data is generated. Figure 1 is the source reconstruction and the sub-circuit taken from it. Source reconstruction: `Oculomotor_sortedData_081126.pkl`, 2949 cells, 42 cell types, 44,584 edges (density 0.51 %). Selected sub-circuit: `config/zebrafish/zebrafish_om_intg_285_v1.yaml`, 285 cells, 8 types.

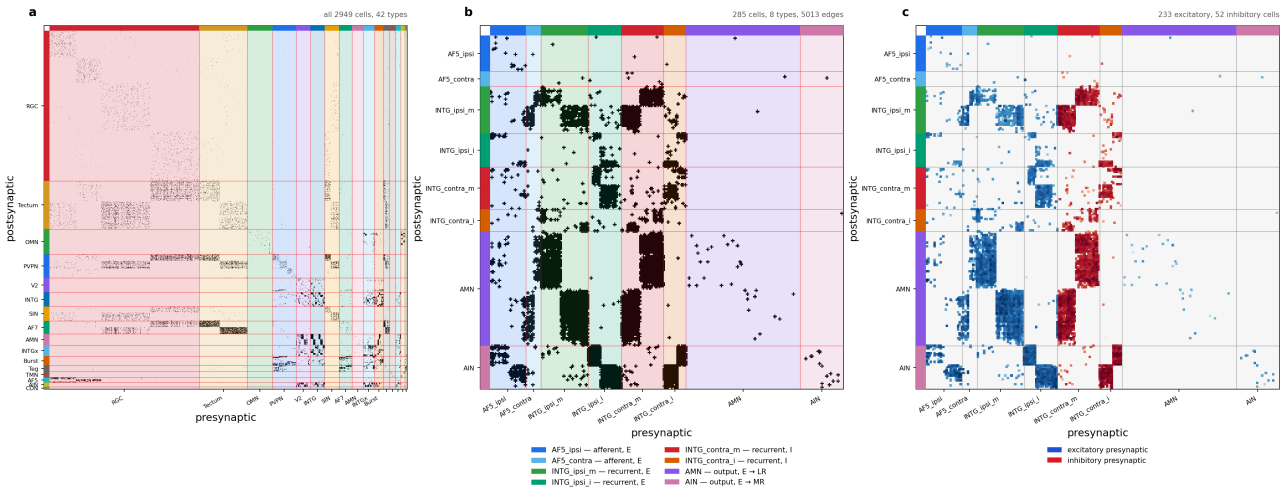

Figure 1: **The oculomotor reconstruction and the sub-circuit modelled here.** (a) All 2949 cells, ordered by the 16 coarse cell-type families, support mask of the synapse-area matrix. (b) The 285-cell sub-circuit, ordered afferent then recurrent then output, and within a type left hemisphere before right. Colour carries the biology: blue = afferent, green = excitatory recurrent, red/orange = inhibitory recurrent, purple/pink = motor output. (c) The same sub-circuit signed by the Dale assignment of section 1.4 — each synapse coloured by the sign of its *presynaptic* cell, blue excitatory, red inhibitory, shade log-scaled in synaptic area. This is what the model multiplies, $\hat{W}_{ij} = |\hat{S}_{ij}|\,\mathrm{sign}(W^{\mathrm{con}}_{ij})$, and the support mask of (b) cannot show it. Because the sign belongs to the column, the panel reads as vertical bands: the two `INTG_contra` types are a solid red block, and it is visibly the only inhibition the 285 cells contain. Rows are postsynaptic, columns presynaptic in all three. Regenerate with `python scripts/plot_oculomotor_connectome.py -config config/zebrafish/zebrafish_om_intg_285_v1.yaml -out figures/zebrafish/fig_oculomotor_circuit.png`.

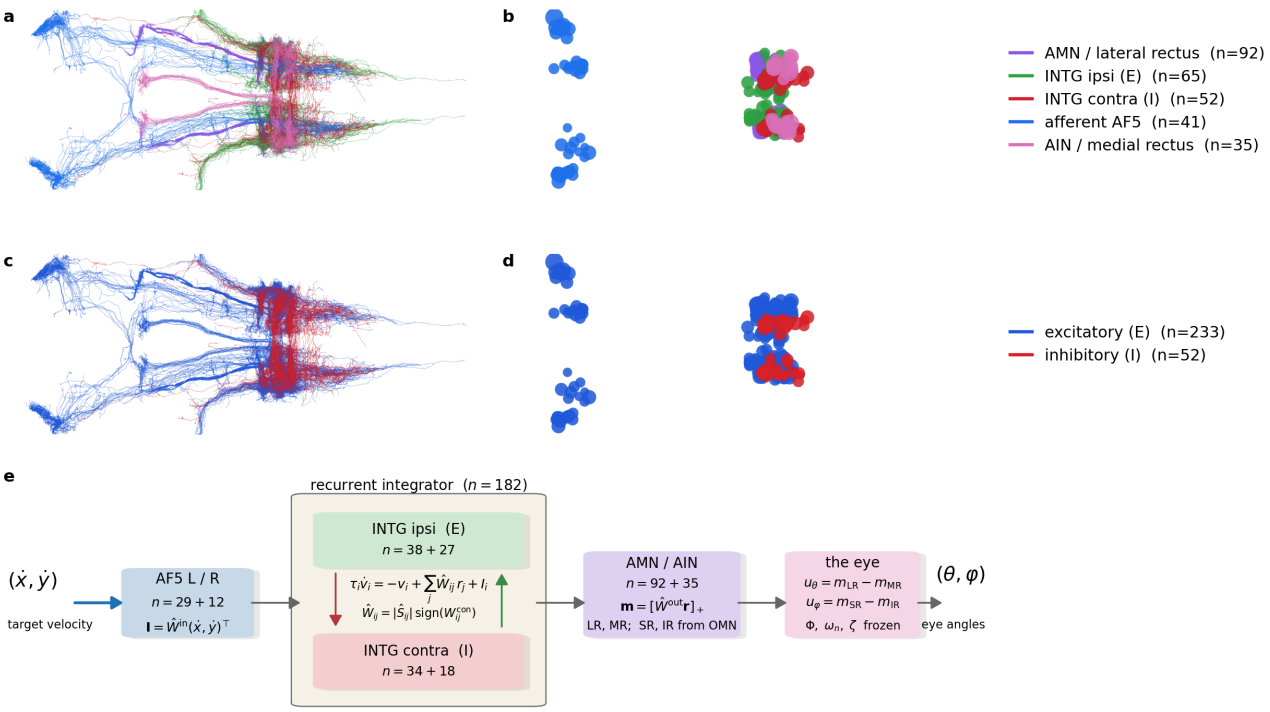

Figure 2: **The oculomotor circuit. (a)** All 285 skeletons from the neuprint-fish2 reconstruction, dorsal view, coloured by role: AF5 afferents blue, ipsilateral (excitatory) INTG green, contralateral (inhibitory) INTG red, AMN purple, AIN pink. **(b)** The cell bodies alone, same view — the AF5 somas sit well anterior of the integrator/motor cluster (radii scaled x0.3 for legibility, not a measurement). **(c, d)** The same skeletons and somas recoloured by Dale sign: blue excitatory (233 cells), red inhibitory (52 cells). The inhibitory population is exactly the contralateral integrator of section 1.4, and the two rows together are the claim being made — that the E/I split is by projection laterality, not by anatomical position. **(e)** The computation of section 4 end to end: the target velocity $(\dot{x}, \dot{y})$ enters through $\hat{W}^{\text{in}}$ on the AF5 afferents, the INTG pool integrates under sign-locked continuous-time rate dynamics with mutual inhibition across the midline, $\hat{W}^{\text{out}}$ reads non-negative motor drives, and the push-pull differences u_θ, u_φ drive the eye to the gaze angles (θ, φ). LR and MR have a pool in these 285 cells; SR and IR would come from OMN, which section 1.5 leaves out. Regenerate with `python figures/zebrafish/fig_1_oculomotor_overview.py`.

1. The circuit

1.1 Three pools

Anatomy for all 285 cells was fetched from neuprint-fish2 and is shown in Figure 2. It had to be keyed on bodyId rather than cell type: the server does not carry the reconstruction's type names — `INTG_ipsi_m` is `INTGip1` there, `INTG_contra_m` is `INTGco1` — and the two AF5 populations carry no type at all, so a type query returns 142 of 285 cells and drops the whole afferent stage. All 285 bodyIds resolve.

The 285 cells divide into an input stage, a recurrent integrator and a motor output stage. Throughout, **the afferents** means the input stage and nothing else — the 41 AF5 cells, the only ones the external drive reaches — and section 1.2 shows that calling them afferent is a measurement rather than a label.

pool	types	cells	Dale sign
afferent	`AF5_ipsi` (29), `AF5_contra` (12)	41	excitatory
recurrent	`INTG_ipsi_m` (38), `INTG_ipsi_i` (27), `INTG_contra_m` (34), `INTG_contra_i` (18)	117	ipsi excitatory, contra inhibitory
output	`AMN` (92), `AIN` (35)	127	excitatory

Within the sub-circuit there are 5013 edges, a density of 6.2 % — an order of magnitude above the 0.51 % of the full reconstruction. That concentration is what one expects of a recurrent integrator core plus its dedicated input and output stages, and it is the first (weak) evidence that the selection is a circuit rather than an arbitrary subset.

1.2 AF5 is an afferent, and the wiring says so

The two AF5 populations are the pretectal arborization field that carries optokinetic drive. Their afferent status is not assumed. In the measured matrix, AF5 sends 88.2 % of its outgoing synaptic area into the INTG/AMN/AIN core and receives 1/178th as much back (6.89e5 out against 3.87e3 in). This is the same asymmetry test that confirmed the matrix orientation using the retinal ganglion cells, which are 15.4x more outgoing than incoming.

Both AF5 types are left/right balanced — `AF5_ipsi` 15 L / 14 R, `AF5_contra` 6 L / 6 R — so a bilateral input gate is expressible. This is worth stating because it is not generally true in this dataset: `RGC_AF7` is reconstructed in the left hemisphere only (804 L / 0 R) and could not support a symmetric gate.

1.3 What drives the afferents — claim, and an open one

The direction selectivity below is the circuit owner's description, not a measurement in this file. The combination rule is explicitly unresolved.

- `AF5_ipsi` — the left pool is driven by rightward-eye motion that is leftward and/or forward; the right pool by leftward-eye motion that is rightward and/or forward.
- `AF5_contra` — the left pool is driven by rightward-eye motion that is rightward and/or backward; the right pool by leftward-eye motion that is leftward and/or backward.

Whether the two conditions combine as AND, OR or XOR is **not known**. This is not a detail to be settled by a default: it decides whether a single afferent unit reports a conjunction (a narrow direction tuning) or a disjunction (a broad one), and therefore how much of the direction computation the recurrent circuit has to do itself. Until it is resolved, the input model should carry it as an explicit switch and the three variants should be trained and compared.

1.4 The integrator, and why the E/I split is by laterality

The four INTG types are the velocity-to-angle integrator. The sign assignment follows their projection laterality: **ipsilateral projections excitatory, contralateral projections inhibitory**. That is the classical integrator motif — two half-integrators, one per hemisphere, each self-exciting locally and inhibiting its opposite number across the midline. Mutual inhibition is what lets the pair hold a *signed* eye angle on a single positive-rate substrate. Panels (c) and (d) of Figure 2 plot the assignment on the anatomy — blue excitatory, red inhibitory — and what they show is that it is not readable from position: the 52 inhibitory cells are interdigitated with the 233 excitatory ones in the same hindbrain column, which is why the split has to be asserted per cell type rather than inferred from where a soma sits.

This is a Dale assignment by cell type, exactly as in the heading-direction circuit, where `_ZHD_INH_PREFIXES` in `connectome_loaders.py` encodes the claim that the r1pi/dIPN cells are GABAergic. Neither is a measurement. The difference is that here the claim is written in the config, per type, where it can be reviewed and flipped, rather than in a tuple of string prefixes inside a loader.

1.5 The output stage, and what it cannot reach

Throughout, **the eye** means everything downstream of the last neuron — the globe, its six muscles, the orbital tissue and their mechanics. What matters is the boundary rather than the word: the circuit's output is a set of muscle drives, the eye turns them into angles, and the eye is measured and frozen while the circuit is learned. The engineering literature, and several of the sources cited later, call this the *plant*; it is the same thing. Section 4.2 identifies it.

The eye takes **six** drives and returns **three** angles — horizontal, vertical and torsion, recovered from the material points by a Kabsch fit (`eye_pose`) — so the readout of step 5 has six rows. The 285 cells fill two of them:

- `AMN` (92 cells) drives the **lateral rectus** (`LR`), which abducts the eye;
- `AIN` (35 cells) drives the **medial rectus** (`MR`), which adducts it.

The other four rows have no source in this pool. SR, IR, SO and IO are innervated by the oculomotor nucleus `OMN`, 207 cells in this reconstruction and not in the selection (section 1.6). This is a gap between the circuit and the eye rather than a limitation of either: the eye is fully six-muscle and the model of section 4 drives it as such, but a circuit built only from these 285 cells can command a strict subset of what the eye can do. Every trained controller in section 5 therefore stands in for the missing rows — it emits all six drives from a free recurrent network — and closing that gap means adding `OMN` to `circuit.cell_types`.

Which four are missing matters. On eye G the two obliques are the torsion pair, SO reaching $+8.7°$ of torsion against 4.0 of vertical and IO -8.2 against 2.9, so the muscles the pool cannot reach are exactly the ones that set ψ. A circuit restricted to AMN and AIN cannot control torsion at all; it can only avoid provoking it.

A second simplification, in the two rows we do have. `AIN` are abducens *internuclear* neurons. In vivo they are not motor neurons: they project across the midline to `OMN`, whose motor neurons innervate the medial rectus. Treating `AIN` as driving MR therefore collapses a two-synapse pathway into one and drops the sign inversion and delay the missing synapse would contribute. That is a legitimate modelling choice, but it should be stated in any write-up, and the control is the same one: add `OMN` and check whether the fitted dynamics change.

1.6 What is not in the pool

Two omissions are deliberate and one of them constrains the task:

- `Burst_T1A/T1B/T8/T9/T10` (74 cells) are the saccadic burst generators — the source of the brief velocity command that a saccadic integrator integrates. They are excluded, so in the current pool the only anatomically defined input is the optokinetic AF5 drive. A saccade-driven task would have to inject velocity directly into INTG, which is a modelling choice, not a connectome-derived pathway.
- `OMN` (207 cells), as described in §1.5.

2. The task

2.1 What the network computes

The oculomotor integrator converts an eye-**velocity** command into an eye-**angle** command: the motor neurons must hold a rate proportional to the angle long after the velocity signal has gone. Written as the leaky integral of the drive, per axis,

$$\frac{d\theta}{dt} = -\frac{\theta}{\tau} + s_\theta\,\dot{x}(t), \qquad \frac{d\varphi}{dt} = -\frac{\varphi}{\tau} + s_\varphi\,\dot{y}(t) \tag{1}$$

where (θ, φ) are the horizontal and vertical eye angles and $(\dot{x}, \dot{y})$ the target velocity — the notation of section 4, in which v is reserved for membrane voltage. The quantity of interest is τ, the time constant over which the eye angle leaks back to centre. A perfect integrator is $\tau = \infty$; the biological integrator is finite, and measuring what τ the connectome-constrained circuit can support is the point of the exercise.

2.2 Saccades versus optokinetic drive

The two natural stimulus regimes differ, and the choice interacts with §1.6:

- **Optokinetic**: sustained whole-field motion, the drive AF5 actually carries. Slow, continuous velocity — the regime the selected pool supports anatomically.
- **Saccadic**: brief high-velocity pulses separated by fixations, the classical integrator probe. This is the regime the burst generators serve, and they are not in the pool.

The swim generator's stimulus is a Poisson train of boxcar impulses (`swim_rate_hz`, `swim_duration_s`) — structurally a saccade train already. So the saccadic regime is reachable by reparameterising the existing generator rather than writing a new one; what is missing is anatomy for the input, not code.

2.3 The open-loop problem

Coarsely: the circuit is asked to keep a moving target centred on the fovea while being told only how fast the target is moving, never where it is. That is the situation the anatomy puts it in. AF5 is a motion-sensitive arborization field — it reports optokinetic *velocity* — and nothing in the selected 285-cell pool reports where the target actually is. A controller with position feedback can always correct, so its errors do not accumulate; a controller given velocity alone must reconstruct the angle by integration and has nothing to check the result against. Drift is therefore not a failure of the circuit but a property of the problem, and the meaningful question is not whether the target is held but for how long.

More specifically, take the horizontal axis and write the target eccentricity $\theta^\star(t)$, the gaze $\theta(t)$, and the retinal error $\varepsilon = \theta^\star - \theta$; the vertical axis is the same with φ. Every trial starts foveated at the centre, $\theta(0) = \theta^\star(0) = 0$. The eye here is idealised as rate control: the motor command sets gaze *velocity*, so gaze angle is its integral. The ideal controller is then

$$\frac{d\theta}{dt} = \dot\theta^\star(t) = s_\theta\, \dot x(t) \qquad \Longrightarrow \qquad \varepsilon(t) = 0 \;\; \forall\, t \tag{2}$$

and tracking is exact. The interesting cases are the three ways a biological integrator departs from it, each of which produces a different growth law for $|\varepsilon|$ — which is what makes them separable from a single measured trace.

Gain. If the velocity-to-angle conversion is miscalibrated by a factor k,

$$\frac{d\theta}{dt} = k\, \dot\theta^\star(t) \qquad \Longrightarrow \qquad \varepsilon(t) = (1-k)\left[\theta^\star(t) - \theta^\star(0)\right] \tag{3}$$

The error is proportional to the *displacement* from the start, not to the distance travelled, because integration is linear. In a bounded workspace displacement is capped by the arena, so this error is self-limiting: in our geometry the target reaches a median 1.16 arena units from the centre, and $|1-k|$ must exceed 0.52 before the target can leave a 0.6-unit fovea at all. A realistic miscalibration of a few percent is invisible. Gain is not what loses the target.

Leak. A real integrator is imperfect and relaxes toward its rest state with a time constant τ:

$$\frac{d\theta}{dt} = -\frac{\theta}{\tau} + \dot\theta^\star(t) \tag{4}$$

In the frequency domain this is a *high-pass* on target eccentricity, corner frequency $1/\tau$. Sustained excursions are under-represented, so the error saturates rather than growing without bound, and gaze is a shrunken, centre-biased copy of the truth. The counterintuitive consequence is that *slow* targets are lost first, because their motion is exactly the low-frequency content the leak removes. Measured: at $\tau = 2$ s a fast target is never lost within 20 s while a slow one goes at 5.9 s, and the slow case needs $\tau \geq 8$ s to survive. τ is the quantity the INTG pool exists to make long, and this is the curve that says how long is long enough.

Noise. If the integrated velocity carries a white perturbation of intensity σ,

$$\frac{d\theta}{dt} = \dot\theta^\star(t) + \sigma\, \xi(t), \qquad \langle \xi(t)\, \xi(t') \rangle = \delta(t - t') \tag{5}$$

the error is a random walk: $\mathrm{std}[\varepsilon(T)] = \sigma\sqrt{T}$ per axis, unbounded but with zero mean. It is the only one of the three that cannot be corrected by better calibration and the only one that averages away across trials, so it is invisible to any analysis that pools trials before measuring drift.

Three growth laws — bounded-linear, saturating, and square-root — over one shared quantity. A drift trace measured from the real circuit can therefore be *classified*, not merely reported, which is the point of setting the problem up this way. `prototype/dot_tracking/openloop.py` implements the gain and leak cases and measures the survival time `t_lose`, the first moment $|\varepsilon|$ exceeds the fovea; the noise case is stated here for completeness but is not in the prototype, since a zero-mean drift is better measured against recorded data than against a simulation of itself.

3. The training dataset

3.1 Where it comes from

Task data is generated by `GNN_Main.py -o generate <config>`, which reaches `data_generate` -> `data_generate_task` (dispatch on `task.task_type`) -> `_generate_swim_integration_task`, all in `src/connectome_gnn/generators/graph_data_generator.py`.

The critical property: **generation is connectome-agnostic**. The generator writes `stimulus.zarr` of shape (trials, T, channels) and `target.zarr` of shape (trials, T, columns) — pure time series, with no notion of a neuron. The circuit appears only as a `circuit_provenance.json` written beside the splits, recording the circuit name, cell count and the SHA-256 of `J_effective`, so a dataset can later be checked against the circuit a checkpoint was trained on.

The consequence for planning: the dataset can be generated and inspected **before** the circuit registry entry, the connectome CSVs or the E/I assignment exist. That is the cheapest next step on this branch.

3.2 Parameters that define it

parameter	meaning
`n_trials_train` / `n_trials_test`	independent trials per split
`n_steps`, `dt`	samples per trial and the timestep, so trial duration is `n_steps * dt`
`swim_rate_hz`	mean Poisson rate of impulse onsets — the saccade rate
`swim_duration_s`	boxcar width of one impulse — the saccade duration
`forward_vel_mean` / `forward_vel_std`	amplitude distribution of the velocity drive
`target_kind`	which target is assembled; `scalar_xi` is the integrator
`xi_tau_s`	the leak; absent or <= 0 gives a perfect integrator
`seed`	stimulus RNG; fixed across baseline and comparison runs

`training.task_targets` then projects the on-disk superset down to the columns a given run trains on, so one dataset serves several sub-tasks.

4. The whole model in one notation

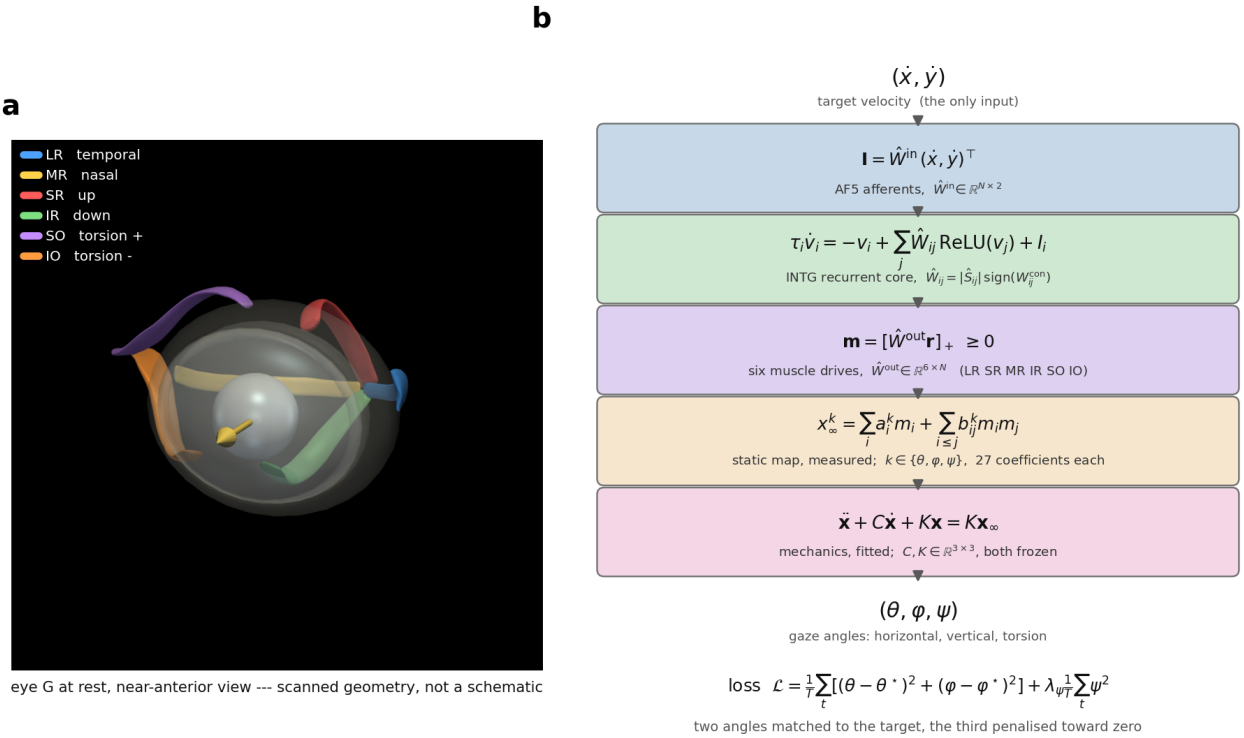

Figure 3: **The eye, and the symbols of the model. (a)** Eye G at rest, near-anterior view, drawn by the surface renderer from the scanned geometry rather than schematised: the translucent globe, the lens, and the six straps as the Blender reconstruction places them. Muscle actions in the legend are the ones stage 0 measured, not the textbook ones — SO elevates and torts positive here, IO depresses, because both obliques pull from the rostral orbit with no trochlea to reverse the superior one. Only the horizontal pair is innervated by this circuit: LR abducts (positive gaze), MR adducts. The four faint muscles exist in the eye but receive no command. The two axes the model uses are marked: horizontal gaze θ, positive in abduction, and vertical gaze φ. **(b)** Every symbol of the nine steps below, in the order the signal meets them: the target velocity $(\dot{x}, \dot{y})$, the input map $\hat{W}^{\text{in}}$ onto the AF5 afferents, the sign-locked recurrent core, the output map $\hat{W}^{\text{out}}$ onto four non-negative motor pools, the push-pull commands u_θ, u_φ, the frozen per-axis eye, and the loss on the gaze angles (θ, φ) — after the eye, not on the command. Regenerate with `python fit_plant.py` and `python fig_eye_schematic.py` in `Plexus/prototype/eye`.

Written in the formalism of `neurips.tex`, with the symbols laid out in Figure 3, so the oculomotor circuit and the flyvis work can be read side by side. Every stage from the two numbers the circuit is given to the two angles it is scored on is written out below, because the intermediate quantities are where the modelling choices are, and they are invisible in any summary that jumps from "velocity in" to "angle out".

symbol	dimension	meaning
$x(t)$, $y(t)$	arena units	target position in the world
$\dot{x}(t)$, $\dot{y}(t)$	units s^{-1}	target velocity — **the only input**
s_θ, s_φ	deg / unit	world-to-angle scale, one per axis
$\theta^\star$, $\varphi^\star$	deg	target eccentricity, horizontal and vertical
$v_i(t)$	—	membrane voltage of neuron i ($N = 285$)
τ_i	s	membrane time constant of neuron i
$r_i(t)$	—	firing rate, $r_i = \rho(v_i)$
$\hat{W} \in \mathbb{R}^{N\times N}$	—	recurrent weights, sign-locked to the connectome
$\hat{W}^{\mathrm{in}} \in \mathbb{R}^{N\times 2}$	—	velocity to afferents; rows zero outside AF5
$\hat{W}^{\mathrm{out}} \in \mathbb{R}^{6\times N}$	—	rates to muscles; columns zero outside the motor pools
$\mathbf{m} \in \mathbb{R}^6$	≥ 0	motor-pool drives, one per muscle
$g = (g^\theta, g^\varphi, g^\psi)$	deg	the **static map**: drives $\to$ the angles the eye settles at
a_i^k, b_{ij}^k	deg	its coefficients, 27 per angle
C, K	—	the eye's mechanics, 3×3 each
θ, φ, ψ	deg	**eye angles** — horizontal, vertical, torsion

A hat marks a learned quantity. v is the membrane voltage throughout and never a velocity; the velocities are $\dot{x}$ and $\dot{y}$, and the eye's orientation is the pair (θ, φ), horizontal first.

Step 0 — the world sets the angles. The target moves in a bounded arena in units, and each axis is mapped to degrees by its own scale, because the eye's reachable travel is not the same horizontally and vertically (section 4.2):

$$\theta^\star(t) = s_\theta\, x(t), \qquad \varphi^\star(t) = s_\varphi\, y(t), \qquad \theta^\star(0) = \varphi^\star(0) = 0 \tag{6}$$

Step 1 — what the circuit is given. Only the velocity, never the position. This is the open-loop premise of section 2.3 written as a vector:

$$\mathbf{u}(t) \;=\; \big(\dot{x}(t),\ \dot{y}(t)\big)^\top \in \mathbb{R}^2 \tag{7}$$

Step 2 — the input map $\hat{W}^{\mathrm{in}}$. The drive enters only through the AF5 afferents $\mathcal{A}$ ($|\mathcal{A}| = 41$). That restriction is the whole content of "connectome-constrained input": a free input layer would let the optimiser inject velocity wherever it is most convenient, which is exactly the degree of freedom the anatomy is supposed to remove:

$$\mathbf{I}(t) \;=\; \hat{W}^{\mathrm{in}}\,\mathbf{u}(t), \qquad \hat{W}^{\mathrm{in}}_{i,:} = \mathbf{0} \quad \forall\, i \notin \mathcal{A}, \qquad \mathcal{A} = \mathcal{A}^L_{\mathrm{AF5}} \cup \mathcal{A}^R_{\mathrm{AF5}} \tag{8}$$

so $\hat{W}^{\mathrm{in}}$ is 285×2 with $41 \times 2 = 82$ free entries out of 570. Componentwise, neuron i receives $I_i = \hat{W}^{\mathrm{in}}_{i1}\dot{x} + \hat{W}^{\mathrm{in}}_{i2}\dot{y}$, and the direction tuning of section 1.3 — still a claim — is what says whether that row should be free or fixed to a preferred direction.

Step 3 — the circuit. The $N = 285$ neurons obey the same rate equation as Eq. (1) of `neurips.tex`:

$$\tau_i \frac{dv_i(t)}{dt} \;=\; -\,v_i(t) \;+\; V_i^{\mathrm{rest}} \;+\sum_{j\in\mathcal{N}_i} \hat{W}_{ij}\, r_j(t) \;+\; I_i(t) \;+\; \sigma\,\xi_i(t), \qquad r_j = \rho(v_j) \tag{9}$$

with $\mathcal{N}_i$ the presynaptic partners of i and ρ the rate nonlinearity — ReLU in the constrained model, tanh in the prototype of section 5.

Step 4 — the sign lock. Only the magnitudes are learned; the signs are the measured Dale assignment of section 1.4, plotted in panels (c) and (d) of Figure 1, so $\hat{W}$ never leaves the connectome's sign pattern:

$$\hat{W}_{ij} \;=\; \big|\hat{S}_{ij}\big|\,\mathrm{sign}\big(W^{\mathrm{con}}_{ij}\big), \qquad \mathrm{sign}\big(W^{\mathrm{con}}_{ij}\big) = \begin{cases} -1, & j \in \mathcal{I} \quad \text{(INTG contra, 52 cells)} \\ +1, & \text{otherwise} \quad \text{(233 cells)} \end{cases} \tag{10}$$

Step 5 — the output map $\hat{W}^{\mathrm{out}}$. The motor pools are read as six non-negative drives, one per muscle, in the order `eye_anatomy` uses:

$$\mathbf{m}(t) \;=\; \big[\,\hat{W}^{\mathrm{out}}\mathbf{r}(t)\,\big]_+ \;=\; \big(m_{\mathrm{LR}},\, m_{\mathrm{SR}},\, m_{\mathrm{MR}},\, m_{\mathrm{IR}},\, m_{\mathrm{SO}},\, m_{\mathrm{IO}}\big)^{\top} \in [0,\infty)^6, \qquad \hat{W}^{\mathrm{out}}_{:,i} = \mathbf{0} \quad \forall\, i \notin \mathcal{M} \tag{11}$$

with $[\cdot]_+$ a rectifier (softplus in the trained controller) enforcing $m_\bullet \geq 0$, and $\mathcal{M}$ the motor pool, so $\hat{W}^{\mathrm{out}}$ is $6 \times N$ with columns live only on motor cells. In the selected 285 cells $\mathcal{M}$ is AMN (92 cells) and AIN (35), which between them innervate LR and MR; **the other four rows have no anatomical pool here**, since SR, IR, SO and IO are driven by OMN, which section 1.5 leaves out of the selection. The six-drive form is written because the eye is six-muscle and because the controller of section 5 emits six; the constrained circuit as currently scoped can fill two of the rows and reaching the rest means adding OMN to `circuit.cell_types`.

Step 6 — the static map g. The six drives set an equilibrium pose $\mathbf{x}_\infty = g(\mathbf{m})$. It is called a *map* and not a gain: it is the function that says where the eye comes to rest for a given set of drives, one component g^k per angle, each a quadratic whose coefficients are measured rather than assumed (section 4.2):

$$x^k_\infty(t) \;=\; g^k\big(\mathbf{m}(t)\big) \;=\; \sum_{i=1}^{6} a^k_i\, m_i(t) \;+\; \sum_{i \leq j} b^k_{ij}\, m_i(t)\, m_j(t), \qquad k \in \{\theta, \varphi, \psi\} \tag{12}$$

27 coefficients per angle, 81 in all. **This is where the push-pull went.** Earlier versions of this model imposed it — one signed command per axis, $u_\theta = m_{\mathrm{LR}} - m_{\mathrm{MR}}$ — so that a signed angle could be carried on non-negative rates. It is no longer imposed, it is an outcome: an antagonist pair shows up as a^k_i of opposite sign, and co-contraction, which no signed scalar can express, shows up in b^k_{ij}. Two of the six muscles turn out to act mainly on ψ, which is why there are three angles here and not two.

Step 7 — the mechanics. The eye swings to that equilibrium under one linear second-order system on all three angles at once, with C and K identified in section 4.2 and **frozen** — registered as buffers, not parameters, because the body is measured and letting the optimiser retune it would defeat the point:

$$\ddot{\mathbf{x}} + C\,\dot{\mathbf{x}} + K\,\mathbf{x} \;=\; K\,\mathbf{x}_\infty, \qquad \mathbf{x} = (\theta, \varphi, \psi)^{\top}, \qquad C, K \in \mathbb{R}^{3\times 3} \tag{13}$$

C and K are full rather than diagonal, so one axis may drag on another, which two independent per-axis plants cannot express.

Steps 6 and 7 are the Hammerstein cascade, the standard first move of block-oriented system identification, and for an eye it is also the classical description — Robinson (1964, 1981), whose linear plant and pulse-step command section 4.1 gives the provenance of. What is new here is only that both blocks are multi-input and multi-output: g takes six drives instead of one signed command, and C, K act on three angles instead of one. The reason the split is worth keeping at that width is the same as at width one — g is memoryless, so (12) can be measured entirely from holds, with no differential equation anywhere in the fit, while (13) is measured from the transients those same holds already contain.

Alternatives, and what the classical model leaves out. Reversing the blocks gives a *Wiener* model, dynamics then nonlinearity, which is the wrong way round here: the nonlinearity is in the muscle and the muscle is upstream of the globe. Wrapping both ends gives Hammerstein–Wiener, and letting the dynamics themselves be nonlinear gives a NARX or neural-ODE eye; both are more general and neither is yet demanded

by a residual of 0.15°. The extension the data does point at is linear-parameter-varying — co-contraction stiffens the eye, so C and K should be scheduled on total drive $\mathbf{1}^\top\mathbf{m}$, which is the one stage of the protocol still outstanding. As for the age of the references, the structure has held: a static muscle nonlinearity into linear orbital mechanics is still the working model and the pulse-step is still how saccadic commands are written. Two of its details have not. Muscle tension relaxes over time scales from tens of milliseconds to seconds rather than the two poles a second-order block allows (Sklavos, Porrill, Kaneko and Dean, 2005), so (13) cannot hold an angle for tens of seconds — tolerable here only because the integration lives in the circuit and not in the eye, which is the whole point of section 5. And rectus paths are constrained by connective-tissue pulleys (Demer and colleagues, from 1995), which eye G has as an operator but not in effect: `muscle_sleeve` is in its spec with stiffness $k = 0$, so the straps run free and their moment arms vary with gaze in a way a real orbit's do not. Both are parameterised through their Cholesky factors, $K = L_K L_K^\top + \varepsilon I$ and $C = L_C L_C^\top$, which makes them positive definite and the eye therefore stable by construction for any values the fit reaches.

Step 8 — the objective. Supervision is on the eye angles *after* the eye, never on the command. Two of the three are matched to the target; the third is penalised toward zero, because six drives against two supervised angles would otherwise leave the network free to wander:

$$\mathcal{L} \;=\; \frac{1}{T}\sum_{t=1}^{T}\Big[\big(\theta_t-\theta_t^\star\big)^2+\big(\varphi_t-\varphi_t^\star\big)^2\Big] \;+\; \lambda_\psi\,\frac{1}{T}\sum_{t=1}^{T}\psi_t^2 \tag{14}$$

The torsion term is Donders' law in its simplest form, and section 5.1 says what λ_ψ is set to and why. Putting the loss after the eye is what makes the network learn the eye's inverse rather than a velocity command: the gradient reaches $\hat{W}, \hat{W}^{\text{in}}, \hat{W}^{\text{out}}$ through the static map and through the mechanics, so a different eye trains a different controller — which section 5.2 turns into a measurement.

Step 9 — discretisation and initial condition. Forward Euler at $\Delta t = 1/60$ s for the circuit, and for the eye the same step with the damping taken *implicitly*:

$$v_i(t{+}\Delta t) = v_i(t) + \frac{\Delta t}{\tau_i}\Big(-v_i(t) + \textstyle\sum_j \hat{W}_{ij} r_j(t) + I_i(t)\Big), \qquad v_i(0)=0,\;\; \mathbf{x}(0)=\mathbf{0} \tag{15}$$

$$\dot{\mathbf{x}} \;\leftarrow\; (I+\Delta t\,C)^{-1}\big[\dot{\mathbf{x}} + \Delta t\,K(\mathbf{x}_\infty-\mathbf{x})\big], \qquad \mathbf{x} \;\leftarrow\; \mathbf{x}+\Delta t\,\dot{\mathbf{x}} \tag{16}$$

The implicit damping is not a refinement. Taken explicitly the rollout is stable only while $\Delta t\,\lambda_{\max}(C) < 2$, and a fit that minimises error at one timestep will walk C up to that edge — ours reached 1.980 at the 9 ms of the recorded holds — and then diverge when the controller rolls the same eye out at 1/60 s, where the product is 3.67. Taking it implicitly removes the limit and, more to the point, makes a fitted C mean the same thing at both timesteps.

$v_i(0) = 0$ has to *mean* "eye centred", which is why every trajectory in the corpus starts at the centre (section 5.1). Nothing is carried between trials.

Two differences from the flyvis setting are worth naming. There the supervision is on dv_i/dt of *every* neuron, because the simulator provides the full state; here it is on two scalars at the far end of an eye, so the circuit is constrained only through its behavioural consequence. And there $\hat{W}$ is what is being recovered from activity, whereas here $\hat{W}$ is fixed in sign by the connectome and only its magnitudes, $\hat{W}^{\text{in}}$ and $\hat{W}^{\text{out}}$ are free. The oculomotor problem is thus the better-posed of the two: fewer unknowns, but a much narrower observation.

What the trained controller of section 5 instantiates. The same nine steps with the connectome removed, which is what makes the anatomy's cost measurable later: $N = 64$ free units in place of the 285 cells, $\rho = \tanh$, $\hat{W}$ dense and initialised at zero with no sign lock, $\hat{W}^{\text{in}}$ a dense 64×2 layer instead of 82 entries on AF5 rows, $\hat{W}^{\text{out}}$ a dense 6×64 layer with all six muscle rows live. Steps 6 to 9 — the measured static map, the frozen 3×3 mechanics, the loss on (θ, φ) with ψ penalised, the Euler step from $v = 0$ — are identical, so the gap between the two is attributable to the anatomy and to nothing else.

4.1 The eye model

The circuit's output is muscle drive, not an eye angle, so the mechanics has to be in the loop — and the MPM eye cannot be: its grid scatter is in-place, so it is not differentiable, and one trial costs more than all 7250 gradient updates of section 5.1. It is replaced by the reduced model of (13), fitted by `fit_plant.py` from the probe runs in `Plexus/prototype/eye/archive/`. Steps 6 and 7 together are the **lateral-horizontal Hammerstein model**: two antagonist pairs, two scalar commands, two independent single-input eyes.

Putting the whole nonlinearity in a memoryless block and leaving the dynamics linear is the **Hammerstein** structure — the standard first move of block-oriented nonlinear system identification (Narendra and Gallman 1966; Bai 1998; Giri and Bai 2010), which dominates practice because it keeps the linear factor a transfer function, with every tool of linear systems theory intact, while the nonlinear factor stays a curve fitted pointwise. For an eye the split is anatomical rather than convenient: the nonlinearity is muscle and memoryless, the dynamics are globe and orbital tissue and near-linear at these rotations. It is also the classical oculomotor model — Robinson (1964, 1981), with later work arguing the real eye carries more time scales than second order allows (Sklavos, Porrill, Kaneko and Dean, 2005).

The factorisation earns its keep here — across mechanical configurations of the MPM eye the mechanics barely move, $\omega_n = 9.5$ to 11.1 rad s^{-1} and $\zeta = 0.22$ to 0.32, while the static gain ranges over 14 to 23 degrees of travel. A configuration changes Φ, not the ODE, so one set of dynamics covers a sweep. At $\zeta \approx 0.3$ the modelled eye is underdamped and rings at about 1.5 Hz; real eyes are overdamped, so that is a property of the MPM configuration rather than of a fish.

Φ and the mechanics are fitted **jointly**, against whole trajectories. The alternative — read the plateaus for Φ, then fit the transient — needs settled plateaus, and this archive has none, for the reason given below.

Second order wins, by about a factor of 3. Against the first-order alternative $\tau\dot{\theta} + \theta = \Phi_\theta(u_\theta)$ — viscosity and an elastic restoring force, no globe inertia — the RMS error is 1.2–2.6 deg at first order against 0.41–0.77 at second, on every configuration that fits at all.

This is not a matter of degrees of freedom. A first-order system is monotone towards its target and *cannot overshoot*, while the MPM eye overshoots and rings. Only the inertial term can produce that. And the fit has to be **per configuration**: pooled it gives RMS 5.05 deg and a static curve with plateaus at +16, +13 and +3 degrees for the same command; fitted separately, 0.41 to 0.77 deg.

What the eye does to a moving target Three traces recur from here on, and Figure 5 is where to see them: **black** the target $\theta^\star$, **red** the command θ_∞ of (12) — the angle the eye *would* settle at if the drives froze — and **blue** the gaze θ of (13), where the eye actually is. (The movies use the same red and blue on a black ground, so the target is white there and black here; it is the same quantity.) At the speeds the task uses red and blue sit almost on top of each other, which looks as though the eye were doing nothing at all. It is not, and what it is doing instead follows from step 7 in three lines.

From the mechanics to a transfer function. (13) is a linear differential equation, so take its Laplace transform. Writing $X(s)$ for the transform of the gaze $\mathbf{x}(t)$ and $X_\infty(s)$ for that of the commanded equilibrium, and starting from rest so the initial-condition terms vanish, $\dot{\mathbf{x}} \mapsto sX$ and $\ddot{\mathbf{x}} \mapsto s^2X$, so

$$s^2X(s) + C\,sX(s) + K\,X(s) \;=\; K\,X_\infty(s) \qquad \Longrightarrow \qquad \big(s^2I + Cs + K\big)X(s) \;=\; K\,X_\infty(s) \tag{17}$$

and the eye is the matrix that carries the command to the gaze:

$$H(s) \;=\; \big(s^2I + Cs + K\big)^{-1}K \qquad \text{(one axis alone: } \; H(s) = \frac{\omega_n^2}{s^2 + 2\zeta\omega_n s + \omega_n^2}\text{)} \tag{18}$$

s is a frequency: $s = i\omega$ asks what the eye does to a command oscillating at ω, and $s = 0$ what it does to one that never changes. Two properties of H are the whole story.

First: it does not shrink the target. Put $s = 0$ in (18) and every term carrying an s vanishes, leaving $H(0) = K^{-1}K = I$: held still, the eye rests exactly at the angle the static map asked for.

That is a property of the model, not a discovery about the eye. The same K on both sides of (13) is a choice: $\ddot{\mathbf{x}} + C\dot{\mathbf{x}} + K\mathbf{x} = B\,\mathbf{x}_\infty$ with $B \neq K$ is just as linear and gives $H(0) = K^{-1}B$. What the choice buys is that $\mathbf{x}_\infty$ *means* the angle the eye settles at, so unity gain at zero frequency is bookkeeping rather than evidence.

The empirical content moves to the static map g of step 6, where it belongs and where it is measured: g is fitted to settled holds, and its residual — 0.15, 0.16 and 0.08 degrees against ranges of 10 to 18 — is the actual statement that this eye comes to rest where g predicts. What could break it is not a wrong gain but a wrong premise: hysteresis, slow drift, or holds that never settle would mean no static map fits at all. That is precisely what the settled criterion (peak-to-peak $\leq 0.05°$) and the protocol's revisit-in-a-different-order check exist to catch, and on eye G all 109 holds passed.

Second: it delays it. Since H is not the identity at nonzero s, ask what it *is* for small s. Factor K out of (18):

$$H(s) = \left(K\left[I + K^{-1}Cs + K^{-1}s^2\right]\right)^{-1}K = \left(I + K^{-1}Cs + K^{-1}s^2\right)^{-1} \tag{19}$$

and expand the inverse, using $(I + A)^{-1} = I - A + O(A^2)$:

$$H(s) \;=\; I - K^{-1}C\,s + O(s^2) \;\approx\; e^{-\Delta s}, \qquad \Delta = K^{-1}C \tag{20}$$

the last step of (20) because $e^{-\Delta s} = I - \Delta s + O(s^2)$ has the same first-order term. Multiplying by $e^{-\Delta s}$ in the frequency domain *is* shifting by Δ in time, so to first order the eye returns its command unchanged, Δ later.

The assumption, stated. (20) is a **low-frequency** expansion and nothing else: it needs $\|K^{-1}Cs\|$ and $\|K^{-1}s^2\|$ both small, which for $s = i\omega$ means

$$\omega \;\ll\; \omega_n \;=\; \sqrt{\lambda_{\min}(K)} \tag{21}$$

— the target must move slowly *compared with the eye's own natural frequency*, not slowly in absolute terms. It is not a large-ω or a small-damping assumption, and it says nothing about how far the eye moves, only how fast. On eye G, ω_n runs 5.8 to 7.3 rad s^{-1} and the approximation holds to 5 % up to 0.26 Hz and to 10 % up to 0.36 Hz, i.e. while $\omega/\omega_n \lesssim 0.3$. Above that it fails in a specific and measurable way, and the table below is where it does.

Measured on eye G. Driving one axis with a small sinusoid and reading the phase and amplitude of the response, against the closed form above:

target	lag θ	lag φ	gain $\lvert H_\theta\rvert$	gain $\lvert H_\varphi\rvert$
0.10 Hz	128 ms	119 ms	1.008	1.006
0.20 Hz	132 ms	122 ms	1.030	1.025
0.35 Hz	143 ms	130 ms	1.095	1.077
0.60 Hz	178 ms	156 ms	1.271	1.221
1.00 Hz	256 ms	220 ms	1.199	1.292
1.50 Hz	255 ms	242 ms	0.564	0.694

with $\Delta = K^{-1}C$ of (20) predicting 127 ms in θ, 118 in φ and 137 in ψ — the low-frequency rows to within a millisecond.

Read it downward. Below about 0.3 Hz the eye is a pure delay of an eighth of a second at unity gain, and red and blue are the same curve shifted by eight frames at 60 Hz (Figure 5a). Between 0.3 and 1 Hz the gain *rises* — rises, not falls, because $\zeta \approx 0.4$ is still underdamped, so the response peaks near ω_n before it rolls off — and the delay grows with it. Past the peak the eye simply cannot keep up: at 1.5 Hz it returns barely half of what it is asked for.

Three things follow for reading the movies. The eighth-second delay is why red and blue separate visibly on eye G — clearest in Figure 5b, where the gaze reaches each new angle a third of a second after the command asks for it — where they nearly coincided on the earlier, faster eye, whose 54 ms was three frames rather than

eight. The rise in gain, not any lag, is what the small loops at direction reversals are: the command turns, the eye carries past it, and the two traces part company for as long as the ringing lasts. And $\Delta = K^{-1}C$ is not a new quantity — it is the same matrix that section 5.2 measures as the controller's *pulse gain*, arrived at there from the trained network rather than from the eye, and the two agree to 8 %.

4.2 Characterising the MPM eye

The lateral-horizontal Hammerstein model of steps 6 and 7 assumes the eye is two independent axes driven by two signed scalars. The measurements say it is not. On eye G the superior oblique produces +8.66 degrees of torsion at full drive against 3.95 of vertical, and the inferior oblique −8.21 against 2.88: torsion is the *largest* action of two of the six muscles. A model with no torsion coordinate cannot represent that, and a model with two commands cannot be told about it. This section is the replacement — the model, the protocol that identifies it, and what eye G measured.

The aim is a procedure rather than a description of one eye. Something will replace eye G; nothing here should need re-deriving when it does. It is executable as `Plexus/prototype/eye/characterise_eye.py`, and `PROTOCOL_eye_characterisation.md` beside it is the prose version.

The eye being characterised Eye G is an MLS-MPM eye whose geometry is **scanned rather than generated**: the globe and the six extraocular straps are filled with material points inside the meshes of a Blender reconstruction of a 96 hpf larval head (`260802_s2_EYE_MUSCLES_MODEL.blend`), by the two seed operators `blend_globe` and `blend_muscles` of `blend_mpm_ops.py`. They are drop-in replacements for the analytic `eye_anatomy` / `muscle_morphogenesis` pair, so the rest of the plant — pose readout, drive, contraction, socket, the shared grid — is model F's mechanics unchanged. What changes is where the tissue is. Its material parameters are model F's throughout: the globe is layered, Young's modulus 45 in the core, 130 at mid-depth and 420 in the sclera, and each of the six straps is elastic at 240 — the *muscles are not stiffness-free*, and their contraction is an active stress on an elastic body, not a kinematic prescription. What *is* set to zero is the connective-tissue sleeve: `muscle_sleeve` appears in the spec with $k = 0$, so the pulleys of Demer and colleagues are declared and inert, and the straps run free between arc fractions 0.70 and 0.88. The orbit holds the globe at $k = 5000$ against fat at 4000, and the tendons anchor to bone at 9000.

Two properties of that geometry matter for everything below, and both were measured rather than assumed. The **insertions are right**: driven in the four cardinal synergies the eye moves where the anatomy says it should, in all four (Figure 4a). And the eye is **contact-limited rather than drive-limited**: scaling the globe by 1.2 about its own centre buries the tendons in the sclera and costs 81 % of the temporal excursion, scaling it by 0.85 collapses the nasal one from 8.5 to 0.7 degrees, and raising the drive ×1.5 makes the globe lose 6.4 % of its radius and returns `NaN` strain. The working point is as hard as this geometry can be driven, which is why the workspace below is taken as given and characterised rather than tuned.

The shape of the model, coarsest first The eye takes six muscle drives and returns three angles:

$$\mathbf{m}(t) \in [0,1]^6 \;\longrightarrow\; \mathbf{x}(t) = \big(\theta(t),\ \varphi(t),\ \psi(t)\big) \in \mathbb{R}^3 \tag{22}$$

horizontal, vertical and torsion, in degrees. The drives are one-sided because muscles pull and do not push, which is why the static map (12) takes them one-sided rather than as the signed differences earlier versions used.

The single structural assumption is that the eye is a **static map followed by linear mechanics**. Where the eye eventually comes to rest is a nonlinear function of the drives; how it gets there is linear:

$$\mathbf{x}_\infty = g(\mathbf{m}), \qquad \ddot{\mathbf{x}} + C\,\dot{\mathbf{x}} + K\,\mathbf{x} = K\,\mathbf{x}_\infty \tag{23}$$

Hold the muscles at a fixed activation and the eye settles somewhere — that is g. Change the activation and the globe swings to the new resting place with an overshoot and a ring — that is C and K. That is (12) and (13) of section 4, and the reason for splitting it that way is given there: g is memoryless, so it can be measured entirely from holds.

The static map, as measured Six inputs is too many to write in closed form and too few for a blind network to fit affordably, so the function is decomposed by interaction order — the functional ANOVA of Hoeffding (1948) and Sobol' (1993):

$$g(\mathbf{m}) = \underbrace{\sum_{i=1}^{6} \phi_i(m_i)}_{\text{one muscle at a time}} + \underbrace{\sum_{i<j} \phi_{ij}(m_i, m_j)}_{\text{pairs}} + \varepsilon(\mathbf{m}) \tag{24}$$

The marginals ϕ_i are measured one muscle at a time (Figure 4b) and the pair terms are screened against the additive prediction $\phi_i + \phi_j$ (Figure 4c). **On eye G the screen refuted additivity outright**: all fifteen pairs exceed the flag threshold, with residuals from 0.09 to 1.03 degrees against a numerical floor of 0.03 — the same hold integrated at two substeps — concentrated in the horizontal axis. The protocol's own stop rule applies at more than eight, so the pairwise grid it prescribes was abandoned rather than expanded: sixty more holds would have bought fifteen bilinear terms and never left the pairwise faces of the cube.

What replaced it is a design over the whole cube. With every pair interacting, the parsimonious form is a quadratic in all six drives,

$$g^k(\mathbf{m}) = \sum_{i=1}^{6} a_i^k \, m_i \; + \; \sum_{i \le j} b_{ij}^k \, m_i m_j, \qquad k \in \{\theta, \varphi, \psi\}, \tag{25}$$

27 coefficients per angle (6 linear, 6 square, 15 cross) and no intercept, because zero drive is the rest pose by construction. It is identified from a scrambled Sobol design of 64 points in $[0,1]^6$ — deterministic, so the design is in the file rather than in a seed — and it fits to 0.15, 0.16 and 0.08 degrees rms in θ, φ, ψ (Figure 4d, and Table 1), against 0.54, 0.62 and 0.47 for the linear map. That is roughly 1 % of each axis's range and five times the numerical floor. Nothing in g is constrained to be monotone; monotonicity is **checked and reported**, not imposed.

That residual carries more weight than a goodness-of-fit number usually does, and section 4.1 says why. Writing the mechanics as (13), with the same K on both sides, forces unity gain at zero frequency — so "the eye comes to rest exactly where it was told to" is true of the *model* by construction and is no evidence about the eye at all. What that construction does is make $\mathbf{x}_\infty$ *mean* the angle the eye settles at, which hands the entire empirical claim to g. These three numbers are it: they are the statement that this eye, held still, rests where g predicts, to about one part in a hundred of its own range.

The mechanics, and what is still open C and K of (23) are fitted from the transients the holds already contain: every hold is a step from rest followed by a release, recorded at full rate, so the 109 runs of Table 1 carry 109 step responses spread across the reachable workspace. Only co-contraction is genuinely missing — two drive patterns matched to the same final pose but different total drive $s = \mathbf{1}^\top \mathbf{m}$ — which is what would say whether the mechanics must be written $C(s), K(s)$. That is three runs, and it is the one block of the protocol still outstanding.

What eye G measured Every stage, what it cost and what it settled:

stage	runs	what it establishes	result on eye G
derisk	4	substep, and the size levers	2.0×10^{-4} within 0.03° of 1.2×10^{-4}; globe ×0.9 and ×0.85 both fail
0-lite	1	the four synergies, and the gate	all four correct; workspace 15.9° h × 24.2° v — passes
0	6	per-muscle span, settling time	LR +7.4 h, MR −6.8 h, SR +7.1 v, IR −8.4 v, SO +8.7 t, IO −8.2 t; $T_{\text{hold}} = 2.84$ s
1	24	the marginals $\phi_i(u)$	strongly convex: LR negative below $u\approx0.2$, SR's gain rises ×107
2a	15	additivity, pair by pair	15/15 non-additive, 0.09–1.03°; the pairwise grid is dropped
6d	64	g over $[0,1]^6$	quadratic rms 0.15/0.16/0.08°; linear 0.54/0.62/0.47°
3	3	co-contraction → $C(s), K(s)$	outstanding
	109		0 unsettled holds

Table 1: The characterisation of eye G, by stage. Every hold keeps its spec, its raw `curves.npz` and a VTK movie in `Plexus/prototype/eye/archive/eye_G/charac/`; `holds.npz` and `report.json` are the assembled forms. Hold length is derived per eye as max(2.0 s, 1.5 × slowest settling) rather than fixed, which is the error that invalidated every earlier characterisation.

What it changed upstream All of this is already in section 4's steps rather than pending: step 5 reads six non-negative drives rather than four, $\hat{W}^{\text{out}} \in \mathbb{R}^{6\times N}$; steps 6 and 7 are the static map and the 3×3 mechanics in place of two independent single-input eyes, with g, C and K frozen buffers; and step 8 carries the torsion penalty, because six drives against two supervised angles leave a four-dimensional set of muscle patterns producing the same gaze. Torsion is not an optional third channel here — SO's largest action is +8.7° of it against 4.0 of vertical, and IO's is −8.2 against 2.9 — so a two-angle model does not describe this eye at all.

When an eye is good enough The same numbers for every eye, reported by the fit: the fraction of holds that settled, held-out rms per axis, the fraction of the command cube where the fitted map is monotone in each muscle's dominant axis, the reachable span per axis against the 15 and 10 degrees the task needs, the eigenvalues of (C, K), and the size of the interaction terms against the marginals. Eye G passes the first four; its interaction terms are large, which is a property of the plant the controller must be trained through rather than a defect of the measurement.

5. Learning the controller

The task is **supervised, not reinforcement**, and it is worth being explicit about why, because the instinct to reach for RL here is strong and wrong. In open loop the correct output is known in closed form at every timestep — given the velocity stream, the target eye angle is its integral. And the environment does not react to the agent: the dot moves the same way whatever the eye does. With no feedback loop and no unknown to explore, this is sequence-to-sequence regression trained by backpropagation through time. RL would recover the same gradient information with far more variance and no compensating benefit.

Closed loop does introduce a loop, but not a reason to change method: the eye is `gaze = integral of command`, which is differentiable, so one simply backpropagates through it. RL earns its place only where the objective stops being differentiable — driving the MPM soft-body eye of `Plexus/prototype/eye` without backpropagating through the simulator, or choosing *when* to make a catch-up saccade, which is a discrete decision rather than a continuous command. Smooth pursuit is regression; saccade timing is a policy.

5.1 How the recurrent models are trained

Everything the controller learns comes from one scalar. It is (14) of step 8, written out again here because it is what every choice below optimises, with $(\theta_t, \varphi_t, \psi_t)$ the gaze the eye actually reaches and $(\theta_t^\star, \varphi_t^\star)$ the target eccentricity of (6):

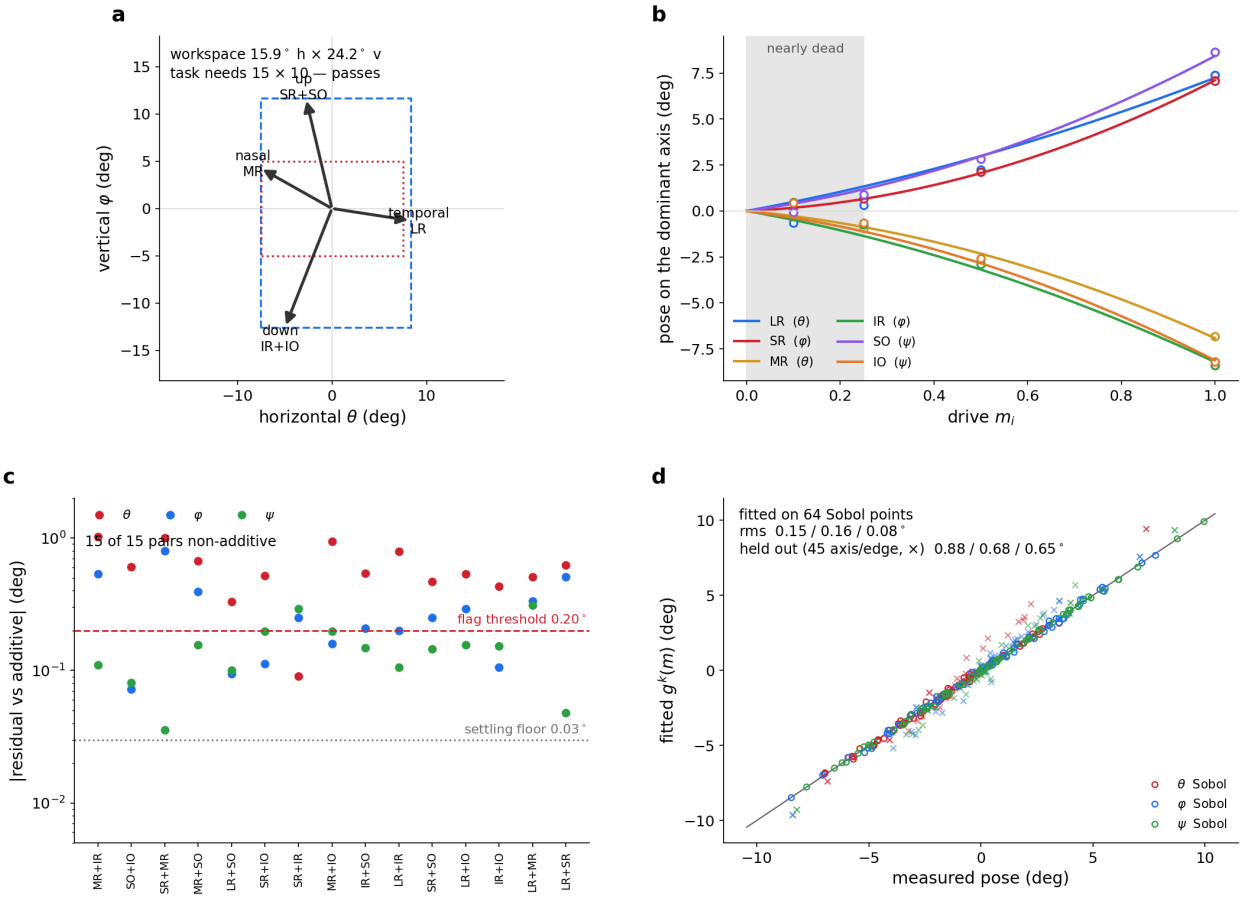

Figure 4: **Eye G, characterised. (a)** Stage 0-lite: the four cardinal synergies driven open loop, each arrow the settled gaze excursion, the dashed box the workspace they span — 15.9° horizontal by 24.2° vertical, against the 15/10 the task needs. All four move the eye in the direction the insertions predict, which is what says the scanned anatomy is right. **(b)** Stage 1: the six marginals on each muscle's own dominant axis. The plant is nearly dead below $u \approx 0.25$ and does most of its work in the upper half of the drive range, so a linearisation about rest is wrong in the direction that matters. **(c)** Stage 2a: the residual of each pair against its additive prediction, per angle, with the flag threshold and the numerical floor. All fifteen pairs interact, mostly in horizontal. **(d)** Stage 6d: the quadratic g of (25) fitted over 64 Sobol points in $[0,1]^6$, measured against fitted. Regenerate with `python Plexus/prototype/eye/fig_eyeG_charac.py`.

$$\mathcal{L} \;=\; \frac{1}{T}\sum_{t=1}^{T}\Big[\big(\theta_t-\theta_t^\star\big)^2+\big(\varphi_t-\varphi_t^\star\big)^2\Big] \;+\; \lambda_\psi\,\frac{1}{T}\sum_{t=1}^{T}\psi_t^2, \qquad \lambda_\psi = 0.05 \tag{26}$$

Note where the angles come from: θ, φ, ψ are the output of (13), so the gradient reaches $\hat{W}$, $\hat{W}^{\text{in}}$ and $\hat{W}^{\text{out}}$ *through* the eye. The eye's own coefficients are buffers rather than parameters and never move. The torsion term is the only part of the loss that is not a tracking error, and it is there because six drives against two supervised angles leaves a four-dimensional set of muscle patterns producing the same gaze; λ_ψ picks one, and picking the untwisted one is Donders' law in its simplest form.

Three choices about how that loss is optimised are load-bearing.

The state starts at $v_i = 0$ for every neuron on every trial, never learned and never carried between trials. That is why the corpus is centre-started: $v_i = 0$ has to *mean* "eye centred", so that the initial condition and the task agree. The loss is dense — every timestep from $t = 0$, not an endpoint — which is what makes the teacher informative at every moment rather than only at the end.

The horizon grows during training, always as a prefix from $t = 0$ rather than a sliding window:

epochs	horizon	
0-62	120 steps	2.0 s
63-124	240 steps	4.0 s
125-186	360 steps	6.0 s
187-249	480 steps	8.0 s

Each stage re-runs from $v_i = 0$ and truncates; there is no truncated BPTT and no carried state. Gradients flow through the whole unroll — 480 sequential steps at the last stage, no detach — with Adam at 2e-3 decayed by cosine to zero, gradient-norm clipping at 1.0, batch 128, so 29 updates per epoch and 7250 in total. The curriculum is not a refinement, it is the reason this trains: at the full 8 s horizon from a cold start the gradient of the integral vanishes, and beginning at 2 s makes the 8 s case reachable. It is the same device as `n_steps_schedule` in the heading-direction configs.

What is trained, and what it reaches. Two controllers, identical but for the eye between them and the width of the readout — four synergy drives against the light eye, six muscle drives against the quadratic one:

	drives	corpus error	test sequence	mean torsion
`eyeG_deep`	6 muscles	**0.055°**	0.10°	0.03°
`eyeG_light`	4 synergies	0.121°	0.24°	0.80°

The corpus column is the 960-trial held-out set, seeds disjoint from training by construction. The test sequence is 48 s of six regimes run continuously — a 5° circle clockwise then anticlockwise, then smooth pursuit and stop-and-go at each of two speeds — joined in velocity so the target never jumps and the state carries across every transition. It is six times the horizon either controller was trained at, and the growth between the two columns is open-loop drift accumulating as section 2.3 says it must.

The six-muscle readout beats the four-synergy one twice over — better tracking *and* a twenty-fifth of the torsion — because with six independent drives the controller can null the torsion against the gaze error instead of trading one for the other. On four synergies it cannot: dropping λ_ψ from 0.05 to 0.005 buys 31 % of the tracking error and costs three times the torsion, which is the trade in plain view.

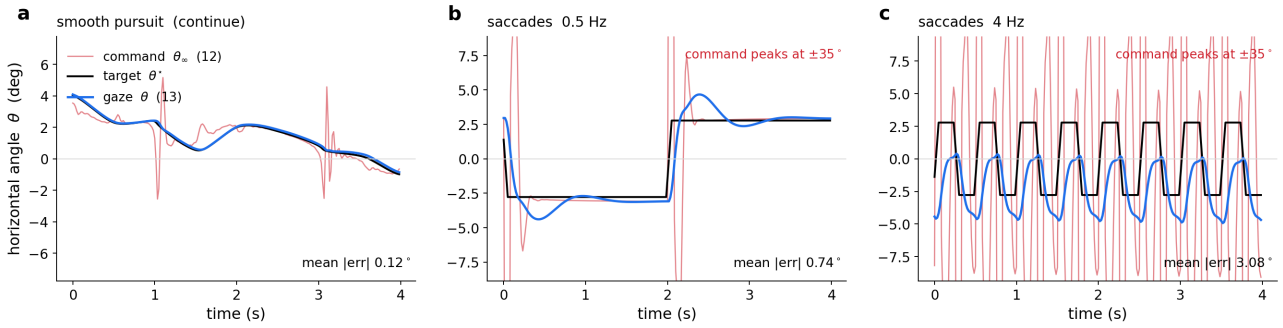

Figure 5: **The six-drive controller on eye G, three regimes.** Horizontal angle only, four seconds of each, drawn from the same rollout the movies play. Black is the target $\theta^\star$, blue the gaze θ of (13), red the command θ_∞ of (12) — the angle the eye would settle at if the drives froze. **(a)** Smooth pursuit: blue sits on black, and red departs from both only at the direction reversals, where the pulse of section 5.2 is needed. **(b)** Saccades at 0.5 Hz: the gaze reaches each new angle in about a third of a second and overshoots slightly, and the command is a doublet reaching ±35° — an order of magnitude beyond the ±2.8° excursion, which is why the axis is scaled to the target and the gaze and the command leaves the panel. **(c)** Saccades at 4 Hz, where the half-period of 125 ms is the eye's own group delay of 127 ms: the gaze is still travelling toward one target when the next arrives, so it never reaches either and tracking has stopped in any useful sense. Regenerate with `python fig_eyeG_traces.py` in `prototype/dot_tracking`.

One result inside the sequence is worth pulling out, because it is a prediction of section 2.3 confirmed on a trained network rather than on an analytic lesion. **Fast targets are tracked better than slow ones**: on the light controller, smooth pursuit at the fast speed gives 0.19° against 0.36° at the middle speed. A leaky

integrator is a high-pass on target angle, so it is the *low*-frequency content that the leak removes, and slow targets are lost first. The six-muscle controller shows the same ordering far more weakly, 0.11° against 0.12°, which is what a better integrator should look like. The circles, a constant-turn regime neither controller has ever seen — the corpus is splines and straight segments — cost nothing beyond that: 0.07 and 0.09° on the six-muscle eye.

Regularisation: almost none, and one that is missing There is no weight decay, no penalty on $\hat{W}$, no spectral normalisation and no dropout. The only two terms that are not the tracking error are the torsion penalty of (26), which is a task constraint rather than a capacity one, and gradient-norm clipping at 1.0, which is a stability device.

That is defensible for capacity: train and validation error agree once put in the same units, so the network was never overfitting and a capacity regulariser would buy nothing. It would matter for **identifiability** rather than for fit — many recurrent matrices integrate, and nothing here selects among them — but in the constrained circuit that job is done by the sign lock (10), so the free-$\hat{W}$ controller is the wrong place to start adding priors.

What *is* missing is a regulariser on the **drives**, and it is not cosmetic. The eye is characterised on $m \in [0, 1]^6$ — every hold in the protocol is at a level of 1 or below — and above that the fitted quadratic simply extrapolates. The trained controllers do not stay inside:

	max drive	fraction of the time above $m = 1$
`eyeG_deep`	1.67	12 %
`eyeG_light`	4.16	**77 %**

The readout is a softplus, which is unbounded above, and nothing in the loss objects. So the light controller spends three-quarters of its time in a regime where the eye model is pure extrapolation, and both leave it entirely on the step probe of section 5.2, where the command reaches 26 degrees against a 7-degree workspace. The real plant is known to behave badly there and not merely to be unmeasured: driving eye G at 1.5 times its working amplitude costs 6.4 % of the globe's radius, takes peak muscle shortening from 26 % to 74 %, and makes two of the four cardinal synergies fail outright.

The fix is a hinge on the drive, $\lambda_m \sum_t \|[\mathbf{m}_t - 1]_+\|^2$, which is free inside the characterised box and grows outside it. It should cost accuracy — the pulse the network needs at a reversal is exactly what pushes m past 1 — and that trade is the point: a controller that tracks to 0.05 degrees by commanding a plant model outside its own validity is reporting a number about the extrapolation, not about the eye.

5.2 The controller rediscovered the pulse-step

Watching the trained network drive the eye turns up something the training never asked for. Whenever the target reverses direction the **command** swings wildly while the **gaze** barely moves off the target: over the sharpest 3 % of turns the gap between command and gaze grows 3.9 times, from 0.56 to 2.18 degrees, while the gaze error grows only 1.33 times, 0.098 to 0.130. The obvious reading is that the motor output has gone unstable and the eye's own smoothing rescues it.

It is not instability. It is the eye's inverse, and the inverse has a closed form, so the claim is falsifiable. Rearranging the mechanics of (13) for the command that would place the gaze exactly on a wanted trajectory $\mathbf{x}^\star(t)$ gives

$$\mathbf{x}^\star_\infty \;=\; \mathbf{x}^\star \;+\; K^{-1}C\,\dot{\mathbf{x}}^\star \;+\; K^{-1}\ddot{\mathbf{x}}^\star \tag{27}$$

Read it in plain terms: to hold the eye somewhere, ask for that place; to move it, ask for somewhere *further*, in proportion to how fast you want to go; and to start or stop the movement, add a term proportional to the acceleration. A step in $\mathbf{x}^\star$ makes the velocity term of (27) a pulse and the acceleration term a doublet, and what is left afterwards is the step. **That is Robinson's pulse-step**, falling out of the algebra rather than being designed in — the same pulse-step section 4.1 credits Robinson (1964, 1981) with, as the command that inverts the linear eye. Nothing in this model was told about it.

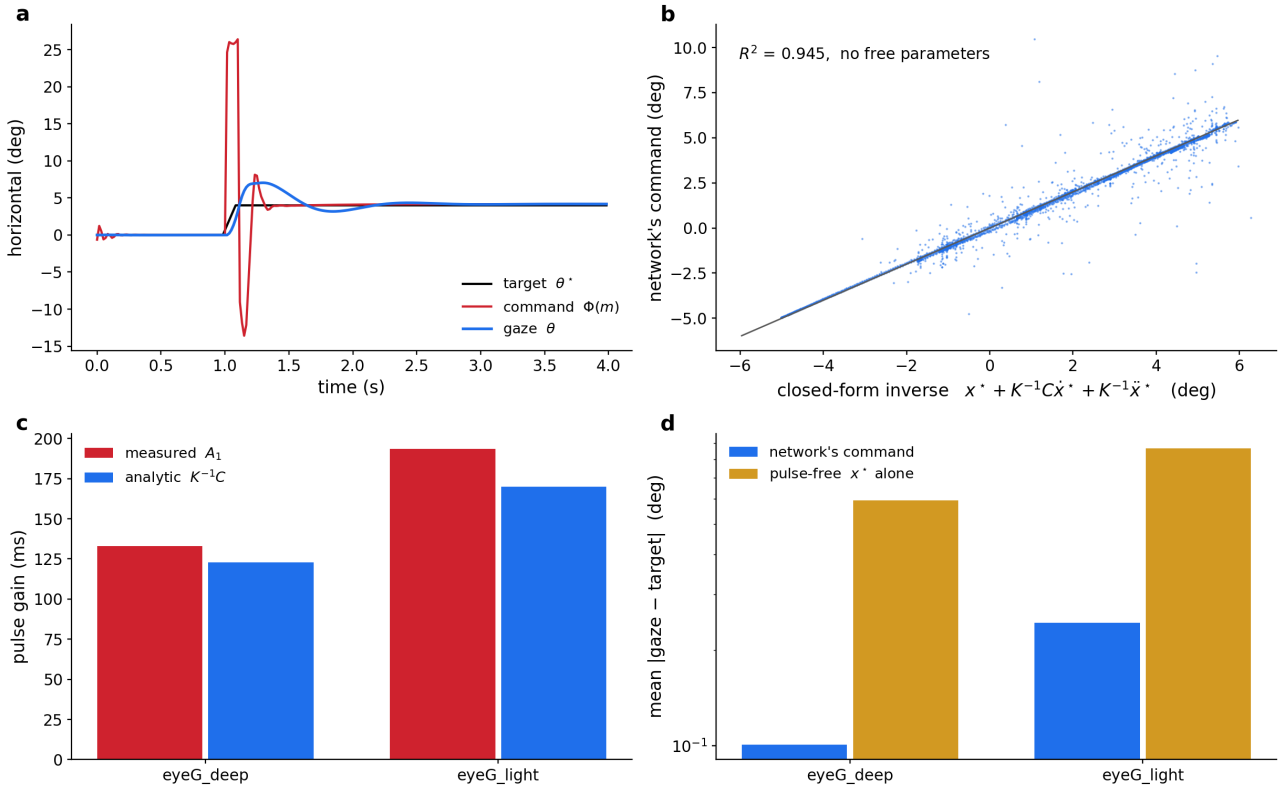

Figure 6: **The controller learned the eye's inverse, not a tracking habit. (a)** A step probe: the target steps 4° and holds. The corpus contains no steps — it is splines at three speeds — so this regime is new to the network. The command goes to +26, brakes to −14, then settles on the +4 step, while the gaze rises smoothly to the target: accelerate, brake, hold. The doublet is the $K^{-1}\ddot{x}^\star$ term and the offset that survives it is $x^\star$. **(b)** The network's command against the closed-form inverse above, evaluated with the eye's own K and C and **no free parameters**: $R^2 = 0.945$. Fitting A_0, A_1, A_2 freely reaches only 0.952, so the analytic matrices cost essentially nothing. **(c)** The velocity gain A_1 of each network against its own $K^{-1}C$. The two eyes differ by 46 % in measured pulse gain (133 against 193 ms) and each matches its own plant; a pulse that was a habit of the architecture or of the corpus would be the same in both. **(d)** Necessity: drive each eye with the pulse-free command $x^\star$ and the tracking error rises 5.9× on the deep eye (0.101 → 0.592°) and 3.5× on the light one. Regenerate with `python fig_pulse_step.py` in `prototype/dot_tracking`.

Why "learned" rather than "built in". Four things have to be true at once, and each is measured in Figure 6.

The shape is not memorised. The corpus is splines at three speeds and contains no steps at all; the step probe of Figure 6a is a regime the network has never been in, and it produces the pulse-step there.

The coefficients are the plant's. Regressing the command freely on the target and its first two derivatives, as (27) would have it, $\mathbf{m}_{\text{cmd}} \approx A_0\mathbf{x}^\star + A_1\dot{\mathbf{x}}^\star + A_2\ddot{\mathbf{x}}^\star$, reaches $R^2 = 0.952$; *replacing* the fitted A_1, A_2 with the analytic $K^{-1}C$ and K^{-1} of (27) — no free parameters at all — still reaches 0.945. The network's velocity gain is 133 ms against the plant's 123. Note the regressor is the **target**, not the achieved gaze: regressing on the gaze would be circular, since the equation above makes that an identity for any command whatsoever.

It is this eye's inverse, not an eye's. The deep and light controllers were trained identically on the same corpus and differ only in the eye between them. Their pulse gains differ by 46 %, 133 against 193 ms, and each tracks its own $K^{-1}C$, 123 against 170. An architectural habit would not know which plant it was attached to.

The loss had no alternative. Driving the same eye with the pulse-free command — just $\mathbf{x}^\star$, no velocity or acceleration term — costs a factor of 5.9 in tracking error on the deep eye and 3.5 on the light one. The pulse is not decoration the optimiser could have skipped.

Two structural points make the result sharper. Nothing in the architecture supplies $\dot{\mathbf{x}}^\star$ or $\ddot{\mathbf{x}}^\star$: the readout is a

static map of the rates, so the velocity term has to come from the input — which *is* the target velocity, scaled — while the angle term must be integrated out of it and the acceleration term differentiated out of it, both by the recurrent dynamics alone. And K and C appear nowhere in the loss; they are buffers inside a frozen eye the gradient passes *through*. The controller inferred them from the only thing it was ever shown, which is how far the gaze ended up from the target.

One last thing worth naming. The recurrent matrix is initialised at exactly zero and every per-neuron time constant is free to grow, so the network could in principle have put this in its membranes. It did not: what it built is an inverse model of the body it is attached to, from a velocity stream and a squared error at the far end of that body, and nothing else.

6. Progress, 11 August 2026

Opened the branch `feat/oculomotor` and put the zebrafish configs back under version control — 254 of them had been absent from every worktree since the `config/zebrafish` bind-mount was commented out of `devcontainer.json`, which left no figure script able to resolve a run config. Traced how the 917-cell heading-direction circuit was actually built (the colleagues' pickle, not the neuPrint query the filenames suggest) and wrote that up as `HOWTO_add_oculomotor_circuit.md`, since the same five inputs are needed again here. Read the new `Oculomotor_sortedData_081126.pkl`: 2949 cells, 42 types, 5 keys, no per-cell ordering variable; confirmed its matrix orientation empirically rather than assuming it. Selected the sub-circuit in two steps — first the 244-cell INTG/AMN/AIN core, then 285 cells once AF5 was added, which gave the pool the afferent stage it had been missing. Introduced `CellTypeSpec` so each type's pool, Dale sign, L/R split and role live in the config where they can be reviewed, rather than in hardcoded prefix tuples inside the loader. Rendered the connectivity figure of Figure 1, and wrote this note.

Nothing has been trained, and no connectome CSVs exist yet. The five decisions below are what stand between this document and a first run.

7. What has to be decided next

1. **The AF5 combination rule** (§1.3) — AND, OR or XOR. Blocks the input model.
2. **Whether `OMN` joins the pool** (§1.5) — decides whether `AIN` -> medial rectus is one synapse or two.
3. **Whether `Burst_*` joins the pool** (§1.6) — decides whether the saccadic regime has an anatomical input or an injected one.
4. **The readout convention** — a scalar eye angle, or innervation of the `LR`/`MR` pair coupled to the soft-body eye.
5. **The target leak `xi_tau_s`** — a free parameter, or the quantity to be fitted against recorded eye angle.

Appendix B. What a contraction is, in the MPM eye

Section 4.2 treats the MPM eye as a black box that turns six drives into three angles. This appendix says what the box does, because two things that decide whether the eye works at all — how hard a muscle may pull, and how its two ends are held — are properties of the discretisation as much as of the anatomy.

The cycle, and where a muscle enters it The plant is MLS-MPM (Hu et al., 2018): material points carry the state, a background grid carries the momentum exchange, and the grid is rebuilt from scratch every substep. One substep of Δt_{sub} — eye G runs $\Delta t_{\text{sub}} = 1.2 \times 10^{-4}$ s inside a frame of $\Delta t = 3 \times 10^{-3}$ s, so 25 substeps per frame — is

1. body forces on the particles, $\mathbf{v}_p \leftarrow \mathbf{v}_p + \Delta t_{\text{sub}}(\mathbf{a}_p^{\text{ext}} - c\,\mathbf{v}_p)$, where $\mathbf{a}^{\text{ext}}$ collects every operator that emits `mpm_acceleration` — the socket, the origin anchor, the sleeve, the drag;
2. stress from the deformation gradient F_p;
3. scatter to the grid (P2G);

4. grid update: momentum to velocity, walls and damping;
5. gather back (G2P): new $\mathbf{v}_p$ and C_p, then $F_p \leftarrow (I + \Delta t_{\text{sub}} C_p) F_p$ and advection.

A muscle enters at step 2 and nowhere else. The elastic stress is fixed corotated, with R_p the polar rotation of F_p,

$$\boldsymbol{\tau}_p^{\text{el}} \;=\; 2\mu_p (F_p - R_p) F_p^{\top} \;+\; \lambda_p J_p (J_p - 1)\, I, \qquad J_p = \det F_p, \tag{28}$$

and `muscle_contract` adds a second term to it, on the muscle's own particles only:

$$\boldsymbol{\tau}_p \;=\; \boldsymbol{\tau}_p^{\text{el}} \;+\; \underbrace{A\, w_{m(p)}\, a_{m(p)}\, T(s_p)\, \big[1 + \beta(\lambda_p - 1)\big]_+\, \mathbf{f}_p \mathbf{f}_p^{\top}}_{\boldsymbol{\sigma}_p^{\text{act}}}. \tag{29}$$

Every symbol is a quantity the model already has: $a_m \in [0,1]$ is the activation of the muscle this point belongs to, integrated by `oculomotor_drive` (or held fixed by `muscle_probe` during characterisation); $\mathbf{f}_p$ is the point's fibre direction, measured by `blend_muscles` as the local gradient of the geodesic fibre coordinate $s_p \in [0,1]$ (0 at the origin, 1 at the insertion); A is the peak active stress (`amplitude`, 67 on eye G); w_m a per-muscle weight, all ones on the scanned plant because the measured cross-sections already carry the relative strengths; $T(s) = \sin(\pi s)^{1/2}$ tapers the drive to zero at both caps, so the tendons are passive and the belly pulls; and $[1 + \beta(\lambda - 1)]_+$ with $\lambda_p = \|F_p \mathbf{f}_p\|$ is the force–length relation, off in eye G ($\beta = 0$).

Why a stress and not a force: the affine tensor The scatter is where this becomes mechanics. Each particle deposits on its 27 neighbouring nodes, weighted by the quadratic B-spline w_{ip}:

$$(m\mathbf{v})_i \;\mathrel{+}=\; w_{ip}\Big[\, m_p \mathbf{v}_p \;+\; \mathbb{A}_p\, (\mathbf{x}_i - \mathbf{x}_p)\Big], \qquad \mathbb{A}_p \;=\; m_p C_p \;-\; \frac{4\,\Delta t_{\text{sub}}}{\Delta x^2}\, V_p^0\, \boldsymbol{\tau}_p. \tag{30}$$

$\mathbb{A}_p$ is a 3×3 *linear map from offset to impulse*, and it holds two things that look unrelated: C_p, the APIC velocity gradient, and the stress. They add because MLS shape functions make $\nabla w_{ip} \propto w_{ip}(\mathbf{x}_i - \mathbf{x}_p)/\Delta x^2$, so the discrete internal force $-\nabla \cdot \boldsymbol{\tau}$ already has the form "matrix times offset". One scatter therefore carries both, and **adding to the stress is adding to the affine tensor**.

Read the active term through (30). Since $\boldsymbol{\sigma}^{\text{act}} = g\, \mathbf{f}\mathbf{f}^{\top}$ is rank one, a node at offset $\Delta\mathbf{x}$ receives

$$-\,\frac{4 \Delta t_{\text{sub}} V^0 g}{\Delta x^2}\, \mathbf{f}\, \big(\mathbf{f} \cdot \Delta\mathbf{x}\big), \tag{31}$$

so a node ahead along $\mathbf{f}$ is pushed backwards and one behind is pushed forwards: both are pulled toward the particle. Summed along a strap the interior cancels and only the two ends survive — a uniaxial tension along the fibre, which is what a muscle is. No force is ever applied to a muscle as an object, and the eye moves because the *divergence* of a stress field reaches the sclera.

One grid, and what attachment means Globe, muscles and (in the pinned-origin variant) bone are separate SETS but share one `mpm_grid`: the stock `mpm_scatter` writes it and the `accumulate` implementation adds to it, so momentum crosses between bodies wherever their particles fall in the same cells. That is the entire contact model, and it has a consequence worth stating plainly: **overlap is a weld**. With the artist's raw geometry, 17 % of SR's points, 21 % of IR's and 11 % of IO's lie inside the retina surface, and the eye barely turns — 5.9/6.0/7.7 degrees against commands of 25/16/9. `blend_muscles` therefore holds the belly a clearance `standoff` $= 0.008$ clear of the globe (the role of Tenon's capsule, over which a muscle slides) and embeds only the distal cap, `embed` $= -0.006$, which is what makes the insertion an attachment rather than a contact.

The proximal cap is the other end and it is not the same kind of object. `bone_anchor` holds the points with $s < 0.10$ to their rest positions with a penalty spring, $\mathbf{a} = k(\mathbf{x}^{\text{rest}} - \mathbf{x}) - c\,\mathbf{v}$, $k = 9000$, $c = 60$. A penalty yields

under exactly the load it exists to resist: on a minimal rig the anchored cap slid 0.063 world units off its bone while the load moved 0.0007, and stiffening does not fix it — at $k = 3 \times 10^5$ the run destabilises before the slip stops. The alternative, implemented as `bone_from_origins` plus `pin_region [clamp]`, gives the origin a *body*: one bone nodule per muscle swallowing its cap, joined to the shared grid and pinned kinematically ($\mathbf{x} \leftarrow \mathbf{x}^{\text{rest}}$, $\mathbf{v} \leftarrow 0$ each substep), so the origin is held the way the tendon is. On the eye it costs 6–19 % of the synergy excursions rather than gaining, because the globe is already held by the socket and the orbital fat; the rig and the eye disagree because they load the attachment differently, and both numbers are kept.

The parameters, and what they are worth Stiffness enters as Lamé parameters, per particle, from a Young's modulus and $\nu = 0.2$: $\mu = E/2(1+\nu)$, $\lambda = E\nu/[(1+\nu)(1-2\nu)]$. Eye G carries sclera $E = 420$, choroid 130, vitreous 45, cornea 320, lens 520, muscle 240, and 1600 for a bone nodule — and the last is not a stiffness to tune: rigidity there comes from the pinning, and the substep caps a usable modulus near 2000 whatever is asked for. Particle volume V_p^0 is the measured mesh volume divided by the points seeded in it, so the six straps differ by a factor of two in V^0 exactly as their scanned cross-sections do; mass is ρV^0 with $\rho = 1$ (2 for bone).

The one dimensionless number that matters is $A/E_{\text{muscle}} = 67/240 = 0.28$: peak active stress against the muscle's own modulus. It cannot be raised much. At $\times 1.5$ the globe loses 6.4 % of its radius, peak shortening goes 26 → 74 % and `strain_p99` returns `NaN`; at $\times 2$ the radius loss is 16 %. The reason is (30): the stress enters scaled by $4\Delta t_{\text{sub}} V^0/\Delta x^2$, so raising A raises the node velocity produced per substep, and past a point the strap deforms faster than the substep resolves. Grid resolution is the other side of the same constraint — 112^3 over the unit box gives $\Delta x = 8.9 \times 10^{-3}$, about 13 cells across the globe's equatorial semi-axis and 2–3 across a strap, which is the coarsest that still separates a muscle from the sclera it slides on.

So the eye is **contact-limited, not drive-limited**, and that is a statement about the discretisation as much as about the anatomy: what bounds the workspace is how much stress the tissue can carry before the substep loses it, and how much of each strap is welded to the globe rather than free to shorten.